软件工程实验三：软件实现与构建

项目名称：多人共享记账系统（合账）
姓名：王紫涵 学号：241220012
课程：软件工程2026秋 实验日期：2026年11月11日
远程仓库：https://github.com/Rekk245/SE-lab/tree/main/lab3

本报告为可修改版本，测试数字和运行输出来自本次实际执行。程序由 Codex 辅助生成和修订；提交前仍需本人逐行阅读源码、检查实现选择并补充个人体会，不能将自动检查结果等同于已经完成人工审核。

1. 实验目的与实现范围

本次实验延续实验一《多人共享记账系统需求规格说明书》和实验二的软件设计。目标是把 UML 中的账本、成员、收支审批、入账、分摊和结算关系实现为可运行程序，使用构建工具生成运行包，并通过 Git 管理本地和远程代码。

本次实现选择 Python，使用 Tkinter 编写桌面界面，使用 SQLite 保存数据。应用运行只依赖 Python 标准库，便于本机演示，也方便后续实验直接调用业务服务进行程序分析和测试。

实验一中多项规则仍标为“待确认”，实验二也用 DD 编号区分了设计候选。因此，本次将相关规则明确写成实现选择，不将其描述为甲方已经确认的需求。数据保留与历史统计的冲突尚未解决，本次保留 UNDECIDED，未执行破坏性清理，不能声称已通过该项最终验收。

1.1 功能范围

功能	本次实现
账本与成员	注册登录、创建与切换账本、创建者邀请、受邀人接受、四角色管理、受约束退出
收支申请	支出金额、类别、日期、报价链接、团队或均摊模式；收入独立待完成审核
状态流转	申请、购买、已经使用、完成；驳回、改价重审、退款核平及撤销
入账与更正	显式完成审核、唯一入账、作废冲减、关联更正申请
分摊结算	整数分均摊、成员或团队垫付、应结债务、线下部分或全部结清
查询统计	类别、申请人、日期及金额组合查询；分类、申请人和均摊承担统计；月/年汇总
预算与日志	月/年预算、超额提示但不拦截申请、按操作人/动作/日期查成功操作日志
工程支撑	手动备份接口、CSV 导出、版本冲突、请求回执、单元测试、构建包、Git 管理

1.2 与实验二设计的差异

实验二组件图采用 Web UI、API Controller、PostgreSQL 和受控附件存储；本次采用本地桌面模块化实现。

原设计	本次实现	影响
浏览器与 API 控制器	Tkinter 页面与事件处理调用业务服务	不需要部署服务器；没有跨电脑联网能力
PostgreSQL 行锁事务	SQLite BEGIN IMMEDIATE 写事务及唯一约束	本地写操作串行化；不宣称具备相同的服务器吞吐能力
凭据链接或文件	仅实现 HTTP(S) 报价链接	满足支出提供报价依据的链接路径，未实现截图上传
保留策略 A/B/UNDECIDED	固定 UNDECIDED，并展示未决说明	不擅自删除，也不承诺长期明细保留合规
多个 Payment 的扩展空间	每笔支出登记一个实际付款主体	支持单人或团队垫付，暂不处理多人共同分次付款

UML/ 保留实验二原始七组 PNG 与 PlantUML 文件，未改写原图。上述部署差异在 UML/图件说明.md 中一并说明。

2. 从 UML 到代码

2.1 分层与模块映射

实现保留了实验二“表现层—业务服务层—数据访问层”的职责划分。界面负责收集输入与显示结果，权限和金额检查统一在服务层执行，数据库层处理事务与关联约束。按钮是否可用不是授权依据；即使绕过界面调用服务，也会重新查询当前账本成员角色。

UML 对象或组件	代码与存储映射
User、Ledger、Membership	auth.py、membership.py；users、ledgers、memberships、invitations 表
Transaction、Expense、Income	domain.py 的不可变数据类；transactions 表按 direction 区分子类型；QueryService.model() 构造对应对象快照
Evidence、Review	evidence、reviews 表；workflow.py 与 base.py 记录报价引用和被审核内容版本
JournalEntry、PostingService	journal_entries 表及 posting.py；POST/REVERSAL 分别唯一
Share、Payment、Obligation	shares、payments、obligations 表；完成支出时在同一事务中产生
OfflineSettlement、SettlementAllocation	settlements、allocations 表；settlement.py 登记实际线下款项
Budget、查询统计组件	budgets 表及 query.py；只汇总有效入账效果
AuditEvent、RequestReceipt	audit_events、request_receipts 表；与业务成功结果共同提交
RetentionPolicy	ledgers.retention_profile 约束为 UNDECIDED；查询接口解释当前范围，不执行清理
六个 UI 页面	ui_design.py 定义主题；ui_pages.py 布局；ui_presenter.py 更新图表与视觉状态；ui.py 处理业务操作

并非每个 UML 类都机械地对应一个带有增删改查方法的 Python 类。纯关系对象映射为数据库表，领域值与收支快照使用数据类，有跨对象一致性要求的行为放入业务服务，避免在多个对象中分别提交事务。

2.2 状态图映射

支出正常流转如下：

```
提交支出 → APPLICATION（申请）
申请批准 → PURCHASE（购买）
登记与获批金额一致的付款 → 仍处于 PURCHASE
申请人确认使用 → USED（已经使用）
完成审核通过 → COMPLETED（完成）+ 唯一入账
```

申请批准不产生费用，登记付款和确认使用也不产生实际统计。只有完成审核成功提交后，总支出才增加。

修改金额、模式或名单时，增加 content_version，清空批准金额及批准内容版本，退回申请。后续付款和完成同时检查批准金额、实际金额与内容版本，旧批准结果不能授权新内容。

申请审核驳回后进入 REJECTED，可修改重提；完成审核驳回的支出保持 USED，保存驳回原因。主动撤销进入 CANCELLED。已完成记录不能当作未完成申请直接修改或撤销，只能由最高管理员通过作废和更正处理。

收入使用 PENDING → COMPLETED，不套用购买与使用阶段。

2.3 活动图与时序图映射

PostingService.complete(transaction_id, version, request_id) 对应实验二完成审核时序图：

1. 开启 SQLite 写事务，查询真实所属账本与当前成员角色。

2. 检查审核权限及普通管理员不得自审的约束。

3. 查询请求回执；同一请求、同一内容返回原入账 ID，同标识不同内容报冲突。

4. 校验显示版本、当前阶段、获批金额、付款金额、内容版本与退款状态。

5. 写审核记录、完成状态和唯一 POST 入账。

6. 对支出生成分摊份额及应结债务，写操作日志和请求回执。

7. 整体提交。任何一步异常都回滚，不保留半完成结果。

SQLite 与原设计的实现机制不同，但仍保留“先校验，状态与财务效果同事务提交”的顺序。数据库还通过外键、非负/范围检查以及 (transaction_id, kind) 唯一约束提供第二层保护。

2.4 金额、分摊与结算

输入金额由十进制字符串转为整数分，拒绝空值、非数字、负数、零、超过两位小数和超出上限的金额。范围 0.01 ~ 99,999,999.99 元是本次采用的候选规则，不是新增的甲方确认事实。

均摊按成员 ID 稳定升序处理余数。例如 100 元由三人分摊，得到 33.34、33.33、33.33 元，合计严格为 100 元。所选成员必须属于账本，不能重复；实际付款人不必是均摊参与者。

团队支出由成员垫付时，生成“团队 → 垫付成员”的债务；团队直接付款不形成报销债务。均摊时，其他参与者向实际垫付人承担各自份额。线下结清只写 settlements 和 allocations 并减少未结余额，不新增 journal_entries，因此不会重复计入费用。

已入账记录作废时保留原 POST，新增 REVERSAL 冲减。原债务未结部分停止生效，已付款部分生成反向退款债务；已有结清记录不重写。更正申请独立审核后产生新的费用和债务。例如团队已报销 200 元，更正为 180 元后，保留 200 元反向应退与新 180 元应付，净额为成员应退团队 20 元，不强行合并历史现金流。

2.5 本次明确采用的候选规则

对应设计候选	实验三选择
DD-01 成员与退出	创建者初始为 OWNER；可有多名 OWNER，至少保留一名；邀请须接受；退出前检查未完成参与和未结债务
DD-02 流转	申请人登记付款、确认使用、退款核平；普通管理员禁止自审；OWNER 仍须显式确认
DD-03 金额、版本、重试	整数分、记录版本与内容版本分开、请求编号与内容摘要配对；不承诺任意未来清理后的重试识别
DD-04 分摊结算	按 ID 分配余分；一个实际付款主体；管理员登记线下结清
DD-05 更正	原记录作废加冲减，创建独立更正申请；已结款通过反向债务调整
DD-06/07 保留	不在 A/B 中擅自选择，清理关闭；本实现仍有保留政策验收缺口
DD-08 预算	按业务日期归属月/年，已完成费用占用预算；未完成申请不占正式预算，超额不阻断

另有一个保守边界：已付款申请若要修改，先登记全额退款到账并核平，再修改、重审和重新登记付款。本版本不处理部分退款，也不覆盖原付款事实来伪装改价。

3. 源代码规模与代码风格

3.1 统计口径

执行 python -m tools.measure，统计范围仅为 shared_ledger/*.py，不包含 tests、tools、run.py、Markdown、UML、运行数据、缓存及构建产物。界面全部手写，没有使用 GUI 设计器或自动生成界面代码。

口径	行数
应用源码物理行	3952
应用源码非空行	3685
排除空行、整行注释及独立文档字符串的代码行	3447
测试代码物理行（单列，不纳入以上数字）	655

SQL 表结构是程序中的可执行模式定义，保留在上述应用源码统计中。不同语言或工具对代码行的定义可能不同，本报告同时提供三种口径和逐文件明细，避免用测试、说明文档或生成文件凑行数。三种应用源码口径均大于 1500 行。

逐文件统计见 evidence/source-size.json，统计工具为 code/tools/measure.py。

3.2 风格约定与检查

以 Google Python Style Guide 为主要依据，采用四空格缩进、80 字符目标行宽、snake_case 函数与变量、CapWords 类名、模块与公开操作文档字符串，以及明确异常类型。使用 Pyink 格式化，使用 Pylint 检查。[1]

实际开发检查版本为 Pyink 26.5.1、Pylint 4.1.3。检查命令：

```
python -m pylint --rcfile pyproject.toml shared_ledger
```

最终输出：

```
Your code has been rated at 10.00/10
```

配置没有完全禁用告警。对 SQL 服务参数数量和界面对象规模设置了合理阈值；界面控件由页面构建方法创建，局部关闭了 attribute-defined-outside-init，注释说明原因。界面布局与事件处理拆成两个模块，收支列表和 CSV 共用格式化函数，避免复制相同显示逻辑。

Pylint 得分表示在本配置下没有报告问题，不等于所有 Google 规范都已经人工验收，更不等于业务正确性证明。完整输出与配置分别见 evidence/pylint.txt 和 code/pyproject.toml。

4. 大模型辅助开发

4.1 实际辅助内容

本次使用 OpenAI Codex 读取实验一需求与实验二报告和 PlantUML，辅助完成模块拆分、应用代码、手写桌面界面、数据库约束、测试、构建工具、风格修订与报告初稿。应用代码主要由模型生成，报告不将模型生成包装为本人已经独立手写或已经逐行审查。

工作先明确业务不变量，再实现服务与事务，随后编写用例验证，最后补齐界面和报告。关键约束包括普通管理员不可自审、完成才入账、改价必须重审、分摊守恒、结清不新增费用，以及未确认保留规则不自动生效。

任务约束的归纳如下，不是伪造的逐字聊天记录：

```
依据已有需求和 UML 实现多人共享记账系统。
先把角色、状态和金额约束落实到服务接口，再写界面。
用整数分处理金额；完成、入账、分摊、日志和回执同事务提交。
测试跨账本访问、管理员自审、重复请求、并发竞争和中途失败。
把候选规则与已确认需求分开；报告只写实际执行过的结果。
```

4.2 生成结果不符合预期时的修订

本次出现的问题	处理与验证
首轮测试的默认临时目录受到本机执行环境限制	为验证过程指定工作目录内的临时位置；未把环境失败写成业务通过
SQLite 初始化连接使用上下文管理器但没有显式关闭	改用 contextlib.closing；业务读写连接始终 finally 关闭，备份恢复测试通过
界面模块过长，列表与导出格式重复	拆出 ui_pages.py，抽取 transaction_cells()；重新检查风格
CSV 文本可能被表格软件当成公式	对全部导出文本字段统一处理公式起始字符，并加入回归用例
格式处理造成部分文档字符串分裂	修复文档字符串，后续格式化对比 AST，检查不引入语义变化
截图工具未能激活目标窗口	不伪造截图，记录真实窗口启动、48 组 Tk 边界及新增交互检查，并说明其局限

这些修订体现了“生成—运行—定位—修正—复验”的过程。自动检查能缩小错误范围，但财务规则是否符合真实团队约定、候选规则是否应该采纳，仍需本人回到需求进行判断。

4.3 本人的审核责任

提交前需要本人逐行阅读即将应用的代码，尤其检查 workflow.py 的状态条件、posting.py 的冲减与债务方向、settlement.py 的余额限制，以及数据库中的唯一约束。应亲自用四种账号完成一次完整流程，并根据自己的理解修改本节体会。本报告没有替本人作出“已经阅读每一行”的声明。

5. 构建与运行结果

5.1 实际验证环境

项目	版本
操作系统	Windows 11，构建号 26200
Python	3.13.7，64 位
SQLite	3.50.4
Tk	8.6
Git	2.53.0.windows.1
应用运行依赖	Python 标准库

应用要求 Python 3.12 或更高版本并带有 Tkinter。当前只在上述 Windows 环境实际验证，不将可移植设计描述为已经完成多平台验收。

5.2 构建方式

```
python -m tools.build
python dist/shared_ledger.pyz --demo
```

构建脚本先使用 compileall 编译检查，再用 zipapp 打包应用源码。[3] 实际输出为：

```
Built shared_ledger.pyz: 44731 bytes
```

该产物是 Python zip 应用，仍需要 Python 与 Tk，不是独立 exe。源码编译检查与打包均成功，且构建产物另在新数据库中执行了无窗口演示，结果与源码版一致。构建记录见 evidence/build.txt 和 evidence/built-demo.json。

5.3 启动与演示数据

在 code 目录执行：

```
python run.py --demo
```

演示账号 owner、admin、member、reader 对应四种角色，密码均为 Demo2026!。初始化只在空数据库执行，不覆盖已有数据。无窗口复现可在全新路径执行：

```
python run.py --headless-demo --db data/fresh-demo.sqlite3
```

演示业务	金额	是否计入实际统计
完成审核的活动经费收入	500.00 元	是
完成审核的小组聚餐，三人均摊	120.00 元	是，每人承担 40.00 元
完成审核的团队物资采购	200.00 元	是
待审活动用品申请	80.00 元	否

实际输出以分为单位：

```
{"income": 50000, "expense": 32000, "net": 18000}
```

因此收入 500 元、支出 320 元、净收支 180 元。月预算设置为 300 元，实际超额 20 元；仍可继续提交申请。均摊费用由普通成员垫付时，其他两名参与者各欠其 40 元；团队费用由该成员垫付时，团队欠其 200 元。报价链接均为演示占位，未进行真实采购或转账。

5.4 界面设计落地与验证

根据实验二六张 UI 设计稿，采用深蓝侧栏、青绿色主按钮、浅灰背景和白色内容卡片。左侧保留六项固定导航，顶部显示当前账本，页面标题与侧栏选中状态同步；表格使用隔行底色和青绿色选中反馈。

页面	对应设计稿的实现
账本与成员	左侧账本列表与成员表，右侧内联创建、邀请及接受邀请卡片
收支管理	左侧组合查询和列表，右侧独立录入表单；双击记录进入审批
申请与审批	顶部四阶段进度，左侧申请事实，右侧审核与申请人操作；收入使用独立两阶段显示
预算与统计	三张收支金额卡、预算进度与超额说明、根据真实入账绘制的分类条形图
分摊与结算	按均摊/团队模式筛选待结款，右侧显示付款双方、未结余额并填入默认结清金额
更正与日志	左侧原金额、冲减与新申请预览，右侧日志过滤和记录列表

图表和预览均读取实际业务数据，不以固定设计稿数字替代查询结果。演示中的食品费用为 120 元、资料费用为 200 元，分类统计与支出 320 元一致。更正预览不会在提交之前改动入账。

程序实际创建了 Tk 窗口。tools.gui_check 分别登录四角色，在默认 1440×940 和最小 1240×820 两种窗口下依次打开六个页面，共 48 组控件边界检查均无越界；还检查了账目选择与阶段显示、更正预览、结算模式筛选和余额默认值，未记录到 Tk 回调异常。

边界与交互检查不替代完整人工可用性测试。桌面截图工具未能激活目标窗口，因此未提供实际运行截图；UI设计参考/ 中六张图片明确是实验二原始设计稿，不冒充运行结果。验证记录见 evidence/gui-check.txt。

6. 测试与关键结果

执行：

```
python -m unittest discover -v
```

实际结果：

```
Ran 50 tests in 146.952s
OK
```

测试组	代表场景与结果
金额与均摊	非法金额均拒绝；100 元三人分币守恒；1～500 分的连续样本均守恒
登录与权限	密码错误拒绝；只读不能提交，普通成员不能审批，管理员不能自审，OWNER 须显式完成
账本隔离	非成员读取、跨账本类别、非账本付款人均被拒绝；登录后降权立即生效
状态约束	不能跳级完成；改价清空批准；付款必须与批准金额一致；不能覆盖付款事实
撤销与驳回	已付款未退款不能撤销；退款核平后可撤销；退款后不能完成入账；完成驳回保持 USED
幂等	同一提交重试十次仍为一笔；同编号不同内容冲突；完成及结清重试不重复计入
事务回滚	在入账后故障、结清分配后故障时整体回滚；重试后金额正确，无半完成结果
结算与更正	部分结清不新增支出；超余额拒绝；200 元已结款更正为 180 元后净应退 20 元
查询与预算	四类条件取交集；空结果正确；预算超额后仍能提交；未完成项不计入实际支出

测试组	代表场景与结果
日志、保留、备份	普通成员不能查日志；未决保留不清理；手动备份重新打开后收支一致
并发一致性	同版本 20 个申请批准竞争只有一个成功；20 个完成竞争只有一次入账
成员规模	创建 200 名不同成员，逐一登录、接受邀请并验证账本读取，成员无丢失或混淆

并发用例使用同一 SQLite 文件上的独立连接和真实线程竞争，不是顺序重复调用。以上说明的是一致性和已确认成员规模；未执行候选的“万条账目、20 并发、95% 请求两秒内成功”性能验收，也未验证定时备份或 30 分钟恢复指标。

完整用例名和输出保存在 evidence/tests.txt。故障注入接点位于入账服务与结清服务的可选构造参数，正常界面不启用。

7. Git 本地及远程管理

本次按本人确认沿用 https://github.com/Rekk245/SE-lab，将新增内容放在 lab3/，保留已有 lab1/。本地以远程 main 分支为基线，只提交实验三新增文件，不改写已有提交。

管理流程为：克隆仓库 → 整理 lab3 代码、UML、报告与证据 → 检查变更 → 提交 → 推送 main → 核对远程提交编号。

```
git clone https://github.com/Rekk245/SE-lab.git
git add lab3
git commit -m "Complete lab3 shared ledger implementation and report"
git push origin main
git ls-remote origin refs/heads/main
```

运行数据库、字节码缓存和临时文件通过 .gitignore 排除，演示账号通过代码生成。仓库中不提交真实业务数据库或外部账号凭据。源代码、测试、构建工具、原 UML、报告与验证证据纳入版本管理；构建产物随提交压缩包提供，可由源码重建。

推送及远程核对的最终结果见 evidence/git-remote.txt；本节只以该文件中的实测结果为准，不将配置远程地址等同于已经上传成功。

8. 尚未完成的范围与后续改进

1. **数据保留未通过最终验收。** 仅金额永久保留与历史分类、年度预算、未结债务的依赖存在冲突；当前保存演示数据并关闭清理，必须先确认永久结构与期限再实现清理和恢复过滤。

2. **桌面程序不提供联网协作。** 四角色和 200 成员是在同一本地数据库的逻辑身份，不是已部署的多人在线服务器。后续可在保留服务层的基础上接回 Web/API 与 PostgreSQL。

3. **附件和付款模型有限。** 仅支持报价链接、单个实际付款主体和全额退款，不支持附件上传或多人分次垫付。系统不执行在线转账，不声称能自动确认实际到账。

4. **备份与性能是局部验证。** 已实现手动备份接口并验证恢复后金额，未实现定时备份、自动恢复或候选性能阈值。

5. **本地安全边界有限。** 应用内重新鉴权不能阻止拥有数据库文件权限的人直接改文件；进程内 Session 也不是远程认证令牌。后续生产化需要重新设计认证与部署边界。

6. **人工审核仍需本人完成。** 自动测试覆盖的是已写入测试的行为，不保证覆盖所有财务场景；提交前应逐行审核并实际操作系统。

9. 提交目录与复现顺序

```
实验3_241220012_王紫涵/
├── 实验报告.md
├── 实验报告.pdf
├── 使用说明.md
├── UML/                    # 七组实验二原图与源码、差异说明
├── UI设计参考/              # 实验二原始六张参考稿
├── code/
│   ├── shared_ledger/      # 应用源码
│   ├── tests/              # 50 项业务测试
│   ├── tools/              # 构建、统计、界面检查工具
│   ├── dist/shared_ledger.pyz  # 随压缩包附带的构建产物
│   ├── run.py
│   ├── build.ps1
│   ├── pyproject.toml
│   ├── 启动演示.bat
│   └── README.md
└── evidence/               # 真实运行和检查输出
```

建议复现顺序：运行 python run.py --demo 查看演示 → 四账号核对权限和流转 → 执行业务测试 → 构建并运行 .pyz → 对照源码规模和风格记录 → 核对远程 lab3。Markdown 修改后，需要重新导出 PDF，避免两份报告内容不一致。

参考资料

[1] Google. Google Python Style Guide. https://google.github.io/styleguide/pyguide.html，访问日期：2026-11-11。

[2] Python Software Foundation. sqlite3 — DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html，访问日期：2026-11-11。

[3] Python Software Foundation. zipapp — Manage executable Python zip archives. https://docs.python.org/3/library/zipapp.html，访问日期：2026-11-11。

[4] SQLite. Transaction. https://www.sqlite.org/lang_transaction.html，访问日期：2026-11-11。用于核对本次 BEGIN IMMEDIATE 写事务与原设计行锁机制的差异。

[5] 王紫涵：《多人共享记账系统需求规格说明书》，实验一提交包 requirements.md，2026-10-25；《软件工程实验二：软件设计与建模》及随附 PlantUML，2026-11-03。本次作为需求和设计输入，原提交包未修改。

[6] OpenAI Codex，本次对话中的代码、测试与报告辅助生成，2026-11-11。使用方式、迭代问题和本人审核责任见第 4 节。